AWS IAM Lab — Groups, Users, Roles & Policies

A Learning Guide: What Was Created, Why, and the Exact Steps to Build It

This document explains the IAM (Identity and Access Management) resources set up for your learning: what each type is, why we created the specific ones we did, the exact step-by-step process used to build them with Terraform, and how you'll use them in the next stages of your data-engineering learning. Companion code: the GitHub repo 'terraform-iam-lab'.

1. The Four IAM Building Blocks (Understand These First)

IAM is how AWS answers one question for every single API call: 'is this identity allowed to do this action on this resource?' There are four building blocks, and confusing them is the #1 source of IAM mistakes.

- **Policy —** the JSON document that lists what is Allowed (or Denied) — which actions (e.g. s3:GetObject) on which resources (e.g. a specific bucket). A policy grants nothing until it's attached to something.
- **User —** a long-lived identity for a person or an application. It can have a console password and/or access keys. You (the human) log in as a user. Permissions come from the groups it belongs to.
- **Group —** a collection of users. You attach policies to the group, and every user inside inherits them. Manage permissions here, not on individual users — it scales and stays consistent.
- **Role —** an identity that is ASSUMED temporarily rather than logged into. It has no password or permanent keys. AWS services (like Glue or Lambda) or other accounts assume a role and get short-lived credentials. This is how services get permissions safely.

One picture ties them together:

```
POLICY (what's allowed) —attached to→  GROUP  —contains→  USER  (a person
logs in)
                        —attached to→  ROLE   ←assumed by—  SERVICE (Glue/
Lambda)
```

Q: User vs Role — what's the one-line difference an interviewer wants?

A: A user is a permanent identity you log in as with long-lived credentials; a role is a temporary identity that a trusted principal (a service or another account) assumes to

receive short-lived credentials. Prefer roles for anything non-human, because temporary credentials are far safer than long-lived keys.

2. What We Created (and Why Each One)

Policies (2)

- **lab-data-engineer-policy —** least-privilege permissions for a data engineer: read/write ONLY the lab S3 bucket, list buckets, and read-only access to Glue/Athena for querying. It does not grant access to every bucket or destructive account-wide actions.
- **lab-analyst-readonly-policy —** a strictly read-only policy for an analyst persona: query data via Athena/Glue and read from S3, but no writes or deletes.

Why custom policies instead of AWS-managed ones like AmazonS3FullAccess? To practice least privilege — granting only what's needed. FullAccess is convenient but over-permissioned; writing your own scoped policy is the real-world skill and a common interview topic.

Groups (2)

- **lab-data-engineers —** has the data-engineer policy attached. Any user added here can work with the lab bucket and query analytics services.
- **lab-analysts —** has the read-only policy attached. Users here can only read/query.

Why groups? So permissions are managed in ONE place. If you later decide data engineers also need Kinesis access, you change the group's policy once, and every member updates automatically — instead of editing ten users.

Users (2)

- **lab-de-learner —** a data-engineer persona, placed in the lab-data-engineers group.
- **lab-analyst —** an analyst persona, placed in the lab-analysts group.

Important: these users were created with NO access keys and NO console password. That's deliberate (see the security section) — credentials are secrets you create separately and carefully, not something to bake into code.

Roles (2)

- **lab-glue-service-role —** a role the AWS Glue service can assume to run ETL jobs. Its 'trust policy' says only glue.amazonaws.com may assume it; its permissions let it use the lab bucket plus the standard Glue baseline.
- **lab-lambda-exec-role —** a role AWS Lambda functions assume: write logs to CloudWatch plus read the lab bucket.

Why roles for services? Because you never want to hand an access key to a running service — it could leak. A role gives the service temporary, auto-rotating credentials instead. When

you build Glue jobs or Lambdas in your next learning steps, they'll assume exactly these roles.

3. The Concept That Trips Everyone Up: Trust Policies

A role actually has TWO policies, and mixing them up is the classic IAM confusion:

- **The trust policy** (assume_role_policy) — says WHO is allowed to assume the role (e.g. 'the Glue service'). Without this, nobody can use the role at all.
- **The permission policies** (the attached policies) — say WHAT the role can do once assumed (e.g. 'read this bucket').

So a working role = 'who can assume me' (trust) + 'what I can do' (permissions). If a Glue job says 'not authorized to assume role,' the trust policy is wrong; if it assumes fine but can't read a bucket, the permission policy is wrong. Knowing which half to look at is a real debugging skill.

Q: A Lambda function gets 'AccessDenied' reading S3. Where do you look?

A: First confirm the Lambda is actually assuming its execution role (trust policy allows lambda.amazonaws.com), then check the role's permission policies include s3:GetObject on that specific bucket ARN. Two different halves — I'd check which stage fails before assuming it's a permissions problem.

4. Exactly How This Was Built (Step by Step)

Everything was done as Infrastructure-as-Code with Terraform — the same approach as your S3 bucket — so it's reviewable, version-controlled, and repeatable. Here's the process, which you can reproduce.

1. **Step 1 — Project setup:** Created a new project folder 'terraform-iam-lab' and defined the Terraform + AWS provider versions (versions.tf) and the provider set to us-west-2 (providers.tf). Note: IAM is a GLOBAL service, so the region only sets the API endpoint — the identities aren't region-specific.
2. **Step 2 — Variables:** Declared inputs in variables.tf: a name_prefix ('lab') so every resource is easy to find/delete, and lab_bucket_arns pointing at your dataengineer-practice-bucket so the policies are scoped to it.
3. **Step 3 — Policies:** Wrote the two customer-managed policies in policies.tf as JSON documents — one least-privilege for data engineers, one read-only for analysts.
4. **Step 4 — Groups:** Created the two groups in groups.tf and attached the matching policy to each.
5. **Step 5 — Users:** Created the two users in users.tf and added each to its group via a group-membership resource (no credentials generated).

6. **Step 6 — Roles:** Created the two service roles in roles.tf — each with a trust policy naming the allowed service, plus permission-policy attachments.

7. **Step 7 — Outputs & safety:** Added outputs.tf to print the created names/ARNs after apply, and a .gitignore so state files and any secrets are never committed.

8. **Step 8 — Validate & publish:** Validated the code locally with 'terraform fmt' (formatting) and 'terraform validate' (syntax/consistency) — both passed — then committed and pushed to a private GitHub repo.

How You Deploy It (the commands)

```
cd terraform-iam-lab
terraform init      # download the AWS provider
terraform plan      # PREVIEW every identity/policy before anything is created
terraform apply     # create them (type 'yes' to confirm)
terraform output    # see the created names/ARNs
# ...later, to remove everything cleanly:
terraform destroy
```

The golden habit: always read 'terraform plan' before 'apply'. It shows you exactly what will be created, changed, or destroyed — for IAM especially, you want to see that before it touches your account.

5. Why No Passwords or Access Keys Were Created

- Terraform stores everything it creates in a state file (terraform.tfstate) in PLAIN TEXT. If Terraform generated an access key or password, that secret would sit in the state file — a real security risk.

- So the users were created WITHOUT credentials. When you actually need to log in as (or authenticate) one of them, you create the credential separately: IAM Console -> Users -> the user -> Security credentials -> create a console password or access key, shown to you once.

- Even better for services: don't create keys at all — have them assume the ROLES instead, which give temporary auto-rotating credentials. That's the modern best practice you're already set up for.

6. Security Best Practices (Interview Gold)

- **Least privilege:** Grant only the permissions actually needed — which is why we wrote scoped policies instead of using FullAccess.

- **Manage via groups/roles:** Attach policies to groups (or roles), not to individual users — it's consistent and scales.

- **Stop using root:** You're currently authenticated as the account ROOT user. Root should be locked away (with MFA) and used almost never. For real work, create an admin IAM user or use IAM Identity Center, and use that instead.
- **Enable MFA:** Turn on Multi-Factor Authentication for root and any human user with meaningful access.
- **Prefer roles over keys:** Prefer temporary role-based credentials over long-lived access keys wherever possible; rotate any keys you do create; never commit them to git or paste them anywhere.

Q: How would you summarize good IAM hygiene in an interview?

A: Least privilege everywhere, permissions managed through groups and roles rather than individual users, root locked away behind MFA and never used day-to-day, temporary role-based credentials preferred over long-lived keys, and everything defined as version-controlled Infrastructure-as-Code so it's auditable and repeatable.

7. How You'll Use These in Your Next Learning Steps

- **Glue jobs:** When you build an AWS Glue ETL job (the real-world version of the PySpark jobs in your food-delivery project), it will assume lab-glue-service-role to read/write your S3 bucket.
- **Lambda functions:** When you write a Lambda (e.g. to trigger a pipeline on an S3 upload), it will run with lab-lambda-exec-role.
- **Practice as a user:** Log in as lab-de-learner (after creating a password/key) to experience working with least-privilege permissions — you'll see firsthand what you can and can't do, which teaches permissions faster than reading about them.
- **Compare personas:** Compare lab-de-learner vs lab-analyst to feel the difference between a read-write and a read-only role.
- **Foundation for everything:** Every AWS data service you learn next (Glue, Athena, Redshift, Kinesis, EMR) is gated by IAM — so this foundation makes all of them make sense.

8. Quick Glossary

- **IAM —** Identity and Access Management — AWS's permission system.
- **Policy —** a JSON document listing allowed/denied actions on resources.
- **User —** a long-lived identity for a person/app, gets permissions via groups.
- **Group —** a collection of users sharing attached policies.
- **Role —** a temporary identity assumed by a service or another account.

- **Trust policy —** the part of a role saying WHO may assume it.
- **STS —** Security Token Service — issues the temporary credentials when a role is assumed.
- **Least privilege —** granting only the minimum permissions needed.
- **MFA —** Multi-Factor Authentication — a second login factor beyond a password.
- **Managed policy —** an AWS-authored reusable policy (e.g. AmazonS3FullAccess) vs a customer-managed one you write yourself.